

University of Göttingen

Thesis Draft

**Rethinking Chain-of-Thought Reasoning through
Diffusion Language Models:
A Comparative Study with Autoregressive
Architectures**

Master's Thesis

submitted to the University of Göttingen

Submitted on:	7 April 2026
Author:	Muhammad Rehan Khalid
Matriculation number:	86949086
Primary supervisor:	David Berghaus (Fraunhofer IAIS)
Second supervisor:	Dr. Caroline Sporleder (University of Göttingen)

Draft generated from Milestone 2 material
University of Göttingen

Declaration of Independent Work

I hereby declare that I have written this thesis draft independently and that all sources, tools, and aids used are stated. I have cited the scientific sources used in this draft according to established academic citation rules. This version is a strengthened thesis draft based on the Milestone 2 material, the local evaluation code, and the available JSON result artifacts.

Göttingen, 7 April 2026

Place, Date

Signature

Abstract

This thesis draft investigates whether chain-of-thought reasoning and planning can be rethought through diffusion language models. It compares LLaDA-8B and Dreamv0-7B with Qwen2.5-7B and Llama-3.1-8B across GSM8K, Countdown-cd4, and Trip Planning. The evaluation combines deterministic accuracy, stochastic pass@k, parser-controlled answer extraction, prompt-mode ablations, and per-question correlation analysis. The central finding is that diffusion language models are stronger in deterministic and low-k planning settings, while autoregressive models improve more under large-k sampling and can overtake at higher sample budgets. Prompt format and few-shot examples substantially affect the ranking, and per-question correlations suggest that the paradigms solve overlapping but meaningfully different subsets of problems. The resulting conclusion is not that diffusion language models simply replace autoregressive models, but that they change the evaluation trade-off and motivate hybrid or ensemble-style reasoning evaluation.

Contents

Declaration of Independent Work	II
Abstract	III
List of Abbreviations	VI
List of Figures	VII
List of Tables	IX
1. Introduction	1
2. Related Work	3
2.1. Autoregressive Language Modeling and Decoding	3
2.2. Chain-of-Thought Reasoning and Sampling-Based Evaluation	3
2.3. Discrete Diffusion and Masked Diffusion Language Models	3
2.4. LLaDA and Dream as Diffusion LLMs	3
2.5. Qwen2.5 and Llama 3.1 as Autoregressive Baselines	4
2.6. Benchmarks: GSM8K, Countdown, and Trip Planning	4
2.7. pass@k Evaluation and Inference Systems	4
3. Model Background and Thesis Hypothesis	5
3.1. Autoregressive Generation	5
3.2. Masked Diffusion Generation	5
3.3. Models Under Study	5
4. Methodology	6
4.1. Benchmark Suite	6
4.2. Models, Checkpoints, and Prompt Modes	6
4.3. Inference Backends and Evaluation Settings	7
4.4. Task-Specific Answer Extraction and Validity Checking	9
4.5. Per-Question Correlation Protocol	10
4.6. Result Provenance	10
5. Results	11
5.1. RQ3 and RQ5: GSM8K prompt sensitivity under a saturating benchmark	11
5.2. RQ1 and RQ2: Countdown-cd4 ranking reversal under pass@k	14
5.3. RQ1 and RQ2: Trip Planning as a natural-language planning check . .	16
5.4. RQ1 control: hyperparameter analysis before paradigm comparison . .	18
5.5. RQ5: Parser sensitivity and failure rates	19
5.6. RQ2: Sampling diversity and pass@k growth	21
5.7. RQ4: Per-question correlation and oracle complementarity	23
6. Discussion	27
7. Limitations	28
8. Conclusion	28

A. Parser Pseudocode	28
A.1. GSM8K Answer Extraction	28
A.2. Countdown Expression Validation	29
A.3. Trip Planning Plan Validation	29
B. Inspected Files and Legacy Parser Notes	30
C. Prompt Templates and Qualitative Examples	30
C.1. Prompt-mode summary	30
C.2. Benchmark-specific answer requirements	31
D. Qualitative Examples	31
D.1. Countdown-cd4 Base	31
D.2. Trip Planning Base	32
D.3. GSM8K parser-sensitivity example	33
D.4. Countdown parser-failure example	33
D.5. Trip Planning parser-failure example	33
References	34
E. AI Usage Card	35

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AR	Autoregressive
CoT	Chain-of-thought
dLLM	Diffusion large language model
GSM8K	Grade School Math 8K
KV	Key-value
LLM	Large language model
NL	Natural language
RQ	Research question

List of Figures

1.	Shared evaluation pipeline for the thesis experiments. Benchmark prompts are generated by dataset-specific code, model outputs are parsed by task-specific extractors, and deterministic accuracy, pass@k, and per-question correlations are computed from the same JSON result format.	7
2.	The parser layer as an explicit measurement stage. Raw completions are routed through task-specific extractors and validators before sample-level correctness vectors, accuracy, pass@k, and correlation metrics are computed. Parse failure, format failure, and validity failure are therefore analytically distinct outcomes.	7
3.	GSM8K pass@k curves across base and instruct settings. The benchmark saturates at high k , but zero-shot instruct behavior differs strongly by paradigm.	12
4.	GSM8K instruct prompt-format sensitivity at pass@1, $n = 128$. Llama-Instruct is especially sensitive to flattening, while LLaDA-Instruct slightly improves under the flat format in this run.	13
5.	Countdown-cd4 base pass@k crossover. Diffusion models lead at pass@1; Qwen and Llama overtake by large k	14
6.	Countdown-cd4 instruct pass@k under templated prompting. Dream-Instruct is the strongest low- k instruct model, while Llama-Instruct becomes strongest at high k	15
7.	Countdown prompt-mode diagnostic from the earlier smaller run. Flat prompting changes Dream and Qwen substantially and leaves Llama-Instruct strong at high k	16
8.	Trip Planning pass@k for all stochastic milestone runs. Dream is strong at low k ; Llama-Instruct flat has the strongest high- k curve.	17
9.	GSM8K sweep summary: best pass@1 and best high- k values. Several diffusion configurations tie at pass@128, so the table reports representative tied configurations.	18
10.	Canonical parser failure rates for the six main comparison conditions. GSM8K and Countdown almost never fail at the parser-entry stage, while Trip Planning can be dominated by malformed outputs.	19
11.	Parser sensitivity diagnostics. Left: GSM8K samples rescued by an alternate final-number fallback. Middle: Countdown alternate expression parser contributes essentially no rescued correct samples. Right: malformed-plan rates remain large for several Trip Planning runs.	20
12.	Mean answer diversity versus pass@k gain. The planning-benchmark points are consistent with a concentration-versus-diversity trade-off: broader answer diversity often accompanies larger gains from search.	21
13.	Mean duplicate rate by model and condition. Several diffusion planning runs repeatedly emit near-identical candidates, while the strongest high- k autoregressive runs keep duplicate rates much lower.	22
14.	pass@k gain by model and condition. The largest gains appear in the autoregressive planning runs, especially Qwen on Countdown Base and Llama on Trip Planning.	22

15.	Summary of per-question correlations. Countdown Base has the strongest dLLM–dLLM agreement, while Trip Planning has the strongest AR–AR agreement.	23
16.	Oracle complementarity gain over the best single model. GSM8K is saturated, but both planning benchmarks show measurable paradigm-level union gains.	25
17.	Question overlap between diffusion and autoregressive unions. Trip Planning instruct has especially large “neither” and nontrivial “dLLM only” / “AR only” regions, which is consistent with complementarity rather than mere redundancy.	26

List of Tables

1.	Research-question-to-evidence map.	2
2.	Benchmark suite and experimental role.	6
3.	Model families and checkpoints.	6
4.	Canonical comparison conditions used in the strengthened draft.	8
5.	Primary result sources used in this draft.	10
6.	Compact cross-benchmark summary of low- k versus high- k behavior. Percentages are computed directly from the canonical JSON result files.	11
7.	GSM8K base 0-shot pass@ k , $n = 128$, temperature = 0.7. Values are decimals from the JSON source.	11
8.	GSM8K base 4-shot pass@ k , $n = 16$, temperature = 0.7.	12
9.	GSM8K instruct pass@ k at 0-shot and 4-shot, $n = 16$, temperature = 0.7.	12
10.	GSM8K instruct templated versus flat pass@ k , $n = 128$, temperature = 0.7.	13
11.	Countdown-cd4 base pass@ k , $n = 128$, temperature = 0.7.	14
12.	Countdown-cd4 instruct-templated pass@ k , $n = 128$, temperature = 0.7.	15
13.	Countdown prompt-mode diagnostic from the earlier smaller run.	15
14.	Trip Planning deterministic comparison	17
15.	Trip Planning pass@ k , merged from Dream/Qwen, refreshed LLaDA, and Llama milestone sources.	18
16.	Best GSM8K sweep configurations recomputed from raw generation JSONs.	19
17.	High-level parser summary. Rates are percentages over all samples in the canonical JSON runs.	20
18.	Compact diversity summary for the main planning conditions. Unique- answer counts are means per question; gains are pass@max minus pass@1 in percentage points.	22
19.	Countdown Base per-question correlations.	23
20.	Countdown Instruct per-question correlations.	24
21.	GSM8K Base per-question correlations.	24
22.	GSM8K Instruct per-question correlations.	24
23.	Trip Planning Base per-question correlations.	24
24.	Trip Planning Instruct per-question correlations.	24
25.	Within-paradigm and strongest cross-paradigm correlations.	25
26.	Oracle complementarity from existing samples. Accuracies are percent- ages of questions solved by at least one sample.	26
27.	Draft AI usage card for the thesis project.	35

1. Introduction

Autoregressive large language models are the dominant reference point for contemporary reasoning and planning systems. Their decoding procedure is simple and powerful: generate the next token conditioned on all previous tokens, repeat until a complete answer is produced, and optionally sample multiple completions when the first answer is uncertain. This left-to-right interface has become the default way to elicit chain-of-thought reasoning, to run few-shot prompts, and to evaluate large language models on mathematical and planning benchmarks [1–4].

This thesis asks whether that default should be rethought through diffusion language models. Diffusion language models do not have to commit to a final answer one token at a time. Masked or discrete diffusion systems initialize a partially masked sequence and refine it over multiple denoising steps, which gives them a different route to global coherence [5–7]. The central question is therefore not only whether diffusion language models can solve reasoning problems, but whether they behave differently enough that the evaluation protocol changes the conclusion.

The empirical evidence in this draft shows that deterministic accuracy alone is insufficient. On the planning benchmarks, diffusion models are stronger in deterministic and low- k settings, but autoregressive models improve faster when many stochastic samples are allowed. In other words, the ranking depends on whether the evaluator asks for one strong solution or permits search over many candidate solutions. This is especially clear on Countdown-cd4, where diffusion models lead at pass@1 and autoregressive models lead at pass@128.

The thesis makes five contributions. First, it provides a controlled comparison of LLaDA, Dream, Qwen2.5, and Llama 3.1 at similar model scale [6–9]. Second, it evaluates the models across GSM8K, Countdown-cd4, and Trip Planning, covering arithmetic reasoning, constrained arithmetic planning, and natural-language itinerary planning [10,11]. Third, it reports both deterministic evaluation and stochastic pass@ k evaluation. Fourth, it uses parser-controlled answer extraction and documents the task-specific validity checks in the evaluation code. Fifth, it adds a per-question correlation analysis showing that failure-mode alignment often depends strongly on generation paradigm, not only on checkpoint family or backbone.

This gap matters because parsers are not implementation details; they are part of the measurement system. A model can produce mathematically correct reasoning but still lose credit if it misses a required boxed answer, emits an invalid Countdown equation chain, or writes a malformed itinerary. Any thesis that compares generation paradigms on reasoning and planning tasks therefore has to treat answer extraction and validity

checking as first-class methodological choices rather than invisible post-processing.

Research questions.

- **RQ1:** Do diffusion LLMs outperform autoregressive LLMs in deterministic or low- k reasoning and planning?
- **RQ2:** Does stochastic pass@ k evaluation reverse the model ranking?
- **RQ3:** How sensitive are the conclusions to prompt mode, chat templates, and few-shot examples?
- **RQ4:** Do diffusion and autoregressive models solve the same questions, or do they have complementary failure modes?
- **RQ5:** How much do task-specific parsers and output-format requirements influence measured performance?

Table 1: Research-question-to-evidence map.

RQ	Claim tested	Main evidence	Caveat
RQ1	Diffusion models may be stronger in single-shot reasoning/planning.	Deterministic comparisons, pass@1 columns, and low- k regions of Figures 5 and 8.	GSM8K is partly saturated and Trip Planning is parser-heavy.
RQ2	Large- k search may reverse the ranking.	Countdown and Trip Planning pass@ k curves, cross-benchmark summary in Table 6, diversity metrics in Figure 12.	pass@ k depends on sample budget and parser-valid output rates.
RQ3	Prompt format and few-shot examples can change the conclusion.	GSM8K 0-shot/4-shot tables, templated-versus-flat comparison in Figure 4, Countdown prompt diagnostic in Figure 7.	Prompt-mode effects mix capability with format mismatch.
RQ4	Paradigms may solve overlapping but different subsets of questions.	Correlation matrices, oracle-union analysis in Table 26, overlap figure 17.	Oracle unions are upper bounds, not deployable ensembles.
RQ5	Parser choice can change measured accuracy.	Parser-layer figure 2, parser-failure heatmap 10, GSM8K sensitivity table 17.	Canonical thesis numbers remain tied to the official evaluation path.

2. Related Work

2.1. Autoregressive Language Modeling and Decoding

Autoregressive language models factor a token sequence into a product of conditional next-token distributions. Transformer decoders made this factorization scalable, and large pretrained autoregressive models established few-shot prompting as a practical evaluation interface [1, 2]. Decoding can be deterministic, as in greedy decoding or low-temperature decoding, or stochastic, as in temperature sampling. The latter is central for pass@k reasoning evaluation because repeated samples can explore alternative solution paths.

2.2. Chain-of-Thought Reasoning and Sampling-Based Evaluation

Chain-of-thought prompting showed that language models can improve multi-step reasoning when prompted to produce intermediate reasoning before a final answer [3]. Self-consistency then showed that sampling multiple chains and aggregating answers can outperform a single greedy chain [4]. This thesis builds on that evaluation perspective, but compares two generation paradigms rather than only two prompts or decoding temperatures.

2.3. Discrete Diffusion and Masked Diffusion Language Models

Diffusion models were originally most visible in continuous domains, but discrete denoising diffusion models extend the idea to categorical state spaces [5]. For language, masked diffusion provides a particularly natural construction: tokens are hidden or corrupted and then iteratively denoised. The model therefore learns to predict missing tokens from bidirectional context instead of only from a left prefix. This difference is the theoretical basis for expecting different reasoning behavior from diffusion and autoregressive language models.

2.4. LLaDA and Dream as Diffusion LLMs

LLaDA and Dream are the diffusion LLMs studied here. LLaDA frames large language modeling as a denoising diffusion process over masked token sequences [6]. Dream is a 7B diffusion language model whose implementation in this repository is adapted from the official Dream codebase and whose planning evaluation code is reused for Countdown and Trip Planning [7]. Dream is especially important for the correla-

tion analysis because it shares architectural lineage with Qwen2.5 while changing the generation paradigm.

2.5. Qwen2.5 and Llama 3.1 as Autoregressive Baselines

Qwen2.5 and Llama 3.1 are strong open autoregressive baselines with base and instruction-tuned variants [8,9]. They are not weak straw models: in the experiments below, they often overtake the diffusion models when many samples are available. Their inclusion therefore makes the comparison a paradigm-level evaluation rather than a diffusion-only ablation.

2.6. Benchmarks: GSM8K, Countdown, and Trip Planning

GSM8K is a standard grade-school math word-problem benchmark used to evaluate multi-step numerical reasoning [10]. Countdown-cd4 is a constrained arithmetic planning task in which the model must compose the supplied numbers to reach a target using arithmetic operations. Trip Planning is a structured itinerary-planning task related to TravelPlanner-style evaluation, requiring ordered city visits and exact stay durations [11]. Together, the benchmarks separate ordinary mathematical reasoning from constrained symbolic planning and natural-language planning.

2.7. pass@k Evaluation and Inference Systems

The pass@k metric, popularized in code generation evaluation, estimates the probability that at least one of k samples solves a problem [12]. It is crucial for this thesis because it exposes a ranking reversal that deterministic accuracy hides. The implementation also matters operationally: diffusion runs use fast-dLLM for the main pass@k experiments, while autoregressive runs use vLLM and its PagedAttention serving system [13]. Standard dLLM and Hugging Face autoregressive runs are retained only for deterministic validation where applicable.

3. Model Background and Thesis Hypothesis

3.1. Autoregressive Generation

For an autoregressive model, a sequence $x = (x_1, \dots, x_T)$ is modeled as

$$p(x) = \prod_{t=1}^T p(x_t \mid x_{<t}). \quad (1)$$

Generation proceeds left to right. Greedy decoding chooses the most likely next token at every step, while temperature sampling draws from a softened or sharpened distribution. Repeated stochastic sampling can therefore explore many different reasoning traces. This is why autoregressive models can be weak at pass@1 but strong at large pass@k: each sample may follow a different path through the solution space.

3.2. Masked Diffusion Generation

Masked diffusion language models begin from a masked or partially masked sequence and iteratively refine tokens over a fixed number of diffusion steps. The practical generation configuration includes generation length, block length, and the number of denoising steps. Instead of committing permanently to each left-to-right token, the model can update a block of positions using broader sequence context. This can concentrate probability mass around globally coherent solutions, which is beneficial for single-sample planning but may reduce the diversity gained from repeated sampling.

3.3. Models Under Study

LLaDA and Dream fit the masked diffusion family. In this repository, diffusion generation is controlled through generation length, diffusion steps, and block length. Qwen2.5 and Llama 3.1 fit the autoregressive family. Their main pass@k runs use vLLM sampling with generation length and temperature, but no diffusion block or denoising schedule.

The thesis hypothesis is therefore: diffusion LLMs may produce stronger single samples because iterative denoising concentrates probability mass around coherent plans, while autoregressive models may scale better under large- k sampling because left-to-right temperature sampling explores more diverse candidate solutions. The results below support this hypothesis on Countdown-cd4 and Trip Planning, while GSM8K is more saturated and mainly reveals prompt sensitivity.

4. Methodology

4.1. Benchmark Suite

Table 2 summarizes the benchmark suite and the role of each benchmark in the thesis.

Table 2: Benchmark suite and experimental role.

Benchmark	Task	Role in the thesis
GSM8K	Grade-school math word problems with final numeric answers.	Reasoning sanity check and prompt/few-shot sensitivity benchmark.
Countdown-cd4	Use four supplied numbers and arithmetic operations to reach a target.	Core constrained-planning benchmark and clearest pass@k ranking reversal.
Trip Planning	Produce an itinerary with ordered cities and exact stay durations.	Natural-language planning benchmark testing whether the Countdown pattern generalizes.

4.2. Models, Checkpoints, and Prompt Modes

Table 3 lists the four model families. Base and instruction-tuned settings are both evaluated whenever available.

Table 3: Model families and checkpoints.

Family	Base checkpoint	Instruct checkpoint	Paradigm
LLaDA	GSAI-ML/LLaDA-8B-Base	GSAI-ML/LLaDA-8B-Instruct	Masked diffusion
Dream	Dream-org/Dream-v0-Base-7B	Dream-org/Dream-v0-Instruct-7B	Masked diffusion
Qwen	Qwen/Qwen2.5-7B	Qwen/Qwen2.5-7B-Instruct	Autoregressive
Llama	meta-llama/Llama-3.1-8B	meta-llama/Llama-3.1-8B-Instruct	Autoregressive

The evaluation code supports three prompt modes. `base_native` is used for base checkpoints and leaves the benchmark prompt in its native flat form. `instruct_templated` applies the model chat template to instruction-tuned models and is the default instruct setting. `instruct_flat` feeds the same task text to an instruct model without chat templating and is used as a prompt-format diagnostic. These modes are implemented in the dataset classes and selected by `inference_and_generation.py`.

4.3. Inference Backends and Evaluation Settings

Figure 1 shows the common evaluation pipeline. Diffusion models use fast-dLLM in the main pass@k experiments. Autoregressive models use vLLM in the main pass@k experiments. Standard dLLM and non-vLLM autoregressive runs are retained for deterministic validation and backend checks, but they are not the main stochastic comparison.

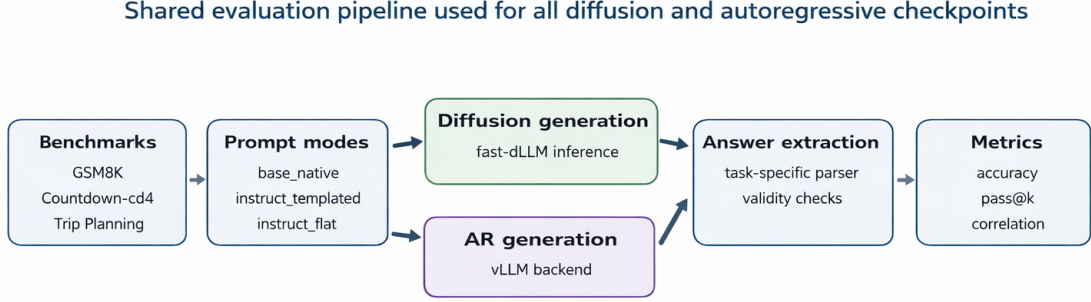


Figure 1: Shared evaluation pipeline for the thesis experiments. Benchmark prompts are generated by dataset-specific code, model outputs are parsed by task-specific extractors, and deterministic accuracy, pass@k, and per-question correlations are computed from the same JSON result format.

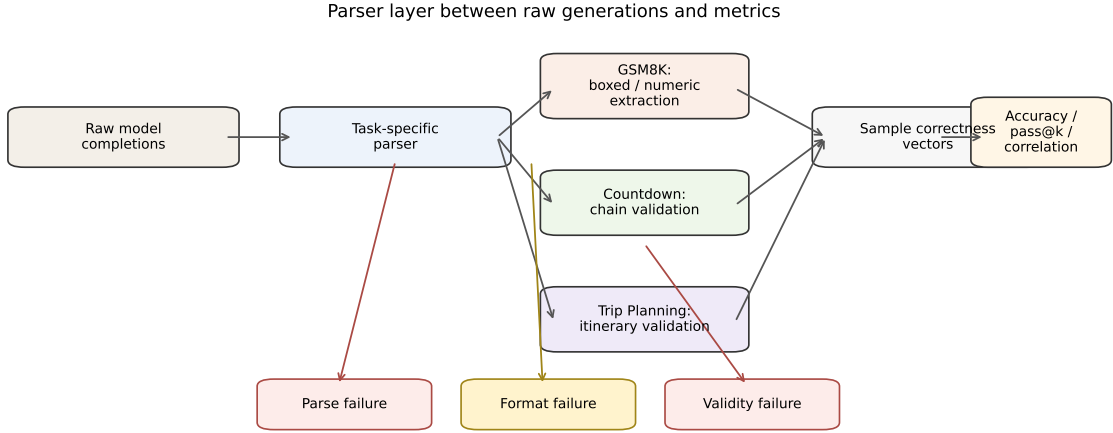


Figure 2: The parser layer as an explicit measurement stage. Raw completions are routed through task-specific extractors and validators before sample-level correctness vectors, accuracy, pass@k, and correlation metrics are computed. Parse failure, format failure, and validity failure are therefore analytically distinct outcomes.

Deterministic evaluation uses temperature = 0 and `n_samples`= 1. Stochastic pass@k evaluation uses temperature = 0.7 and `n_samples`= 16, 64, or 128 depending on benchmark and run. For a problem with n generated samples and c correct samples,

the unbiased pass@k estimator used by the repository is

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \quad (2)$$

The implementation is in `metrics/pass_k.py` and the comparison script is `evaluate_pass_k.py`.

Table 4: Canonical comparison conditions used in the strengthened draft.

Condition	Questions	Few-shot	n samples	Temp.	Notes
GSM8K base comparison	128	4	128	0.7	<code>base_native</code> ; 256-token generations.
GSM8K instruct comparison	128	4	128	0.7	<code>instruct_templated</code> plus <code>instruct_flat</code> diagnostic.
GSM8K 0-shot / 4-shot ablations	256	0 or 4	16	0.7	Used to isolate few-shot effects.
Countdown-cd4 base / instruct	992	8	128	0.7	32-token generations; diffusion runs keep denoising-step and block-length settings in filenames.
Trip Planning base / instruct	200	2	64	0.7	256-token generations; merged from refreshed LLaDA/Llama plus earlier Dream/Qwen comparison files.
Deterministic validation	benchmark-dependent	matched to task	1	0	Used for pass@1-only backend checks.

For reproducibility, `inference_and_generation.py` sets `torch`, `numpy`, and `Python`

random seeds to 2 before dataset construction. GSM8K and Trip Planning additionally use fixed `RandomState(42)` subsampling inside the dataset classes; Countdown draws its 992-item evaluation subset after the global NumPy seed is fixed. Prompt templates are defined in `dataset/gsm8k.py`, `dataset/countdown.py`, and `dataset/trip_planning.py`. The exact generation length, diffusion-step count, block length, batch size, few-shot count, evaluation-set size, and sample count are encoded in the result filenames and summarized in the audit file.

4.4. Task-Specific Answer Extraction and Validity Checking

The parser layer is part of the experimental design. Different benchmarks require different validity checks, and the thesis results should be read together with these extraction rules.

GSM8K. The GSM8K dataset loader extracts gold answers from the official answer text using the `####` final-answer marker and a numeric regular expression in `Parser.extract_answer_gsm8k`.¹ Generated answers in the main evaluator are extracted with `Parser.extract_answer_boxed`, which takes the last `\boxed {...}` or `\fbox {...}` expression found in the model output. The extracted string is converted to a float when possible. The gold parser removes commas in official numeric answers; the boxed-output parser returns the boxed content and relies on float conversion for numeric normalization. In the main `pass@k` path, there is no final-number fallback for GSM8K model outputs. Correctness is computed by comparing the extracted value with the gold value, with numeric tolerance in `evaluate_pass_k.py`.

Countdown-cd4. Countdown scoring uses the Dream-derived chained-equation metric in `dataset/countdown.py`.² The main evaluator takes the first output line as the prediction and calls `cd_score_single`. The scorer normalizes common operator variants such as `\times`, `\cdot`, `\div`, and Unicode multiplication/division symbols. It then extracts leading comma-separated binary equations of the form `a op b = c`. Each left-hand expression is evaluated with restricted `eval`; malformed expressions are rejected. A multiset counter enforces that the supplied numbers are used exactly as allowed, intermediate results are introduced only by previous equations, and the final right-hand side equals the target. The allowed arithmetic operators are addition, subtraction, multiplication, and division in their normalized forms.

Trip Planning. Trip Planning scoring uses the Dream-derived parser in

¹Main files inspected: `dataset/gsm8k.py`, `metrics/parsers.py`, `inference_and_generation.py`, and `evaluate_pass_k.py`.

²Main files inspected: `dataset/countdown.py`, `inference_and_generation.py`, and `evaluate_pass_k.py`.

`dataset/trip_planning.py`.³ The parser scans the generated text for a total-day statement, visit day ranges, and flight lines of the form “Day d ... from City to City”. It reconstructs the ordered city sequence and stay lengths from the flight days. The scorer compares this parsed plan with the gold ordered city list and gold durations. It is a strict validity-style exact match over the required prefix: malformed plans parse to an empty list and therefore fail. Legacy JSON-style Trip Planning helpers exist in `utils.py`, but they are not used by the main result JSONs analyzed here.

4.5. Per-Question Correlation Protocol

For each benchmark condition, the analysis computes a per-question accuracy vector for each model. Each vector entry is the fraction of samples for that question that were correct. Pearson correlations are then computed between model pairs. The canonical comparison uses 128 samples per model for GSM8K and Countdown, and 64 samples per model for Trip Planning. The HTML comparison files generated by `generate_comparison_html.py` provide per-question inspection views for base and instruct settings.

4.6. Result Provenance

The thesis tables and figures use the following JSON result files as source material.

Table 5: Primary result sources used in this draft.

Result group	Source files
GSM8K pass@k	Milestone 2 GSM8K comparison JSONs for base, base 4-shot, instruct, instruct 0-shot, and instruct 4-shot. Exact paths are listed in the audit file.
Countdown pass@k	Refreshed Countdown base/instruct comparison JSONs plus the earlier prompt-mode diagnostic JSON.
Trip Planning	Deterministic comparison JSON, Dream/Qwen large-k JSON, refreshed LLaDA pass@k JSON, and Llama pass@k JSON.
Hyperparameter sweep	Raw GSM8K generation JSON directories for LLaDA-Base, Dream-Base, and Qwen-Base.
Correlation analysis	Six generated 4-model HTML comparison files and the corresponding generation JSONs listed in <code>generate_comparison_html.py</code> .

³Main files inspected: `dataset/trip_planning.py`, `inference_and_generation.py`, and `evaluate_pass_k.py`.

5. Results

Table 6 compresses the main empirical story before the benchmark-specific sections unpack it in detail.

Table 6: Compact cross-benchmark summary of low- k versus high- k behavior. Percentages are computed directly from the canonical JSON result files.

Condition	Best dLLM at pass@1	Best AR at pass@1	Best dLLM at pass@max	Best AR at pass@max
GSM8K base	Dream 72.39	Qwen 69.51	Dream 97.66	Qwen 100.00
GSM8K instruct	Dream 76.15	Qwen 79.77	LLaDA 99.22	Qwen 100.00
Countdown base	LLaDA 14.29	Qwen 4.04	Dream 37.60	Qwen 51.31
Countdown instruct	Dream 9.15	Qwen 7.84	Dream 32.26	Llama 46.17
Trip Planning base	Dream 5.44	Llama 2.45	Dream 22.50	Llama 25.50
Trip Planning instruct	Dream 5.66	Llama 1.71	Dream 15.00	Llama 21.00

moder

5.1. RQ3 and RQ5: GSM8K prompt sensitivity under a saturating benchmark

RQ3 asks how much the conclusion depends on prompting, and GSM8K gives the cleanest answer: the benchmark is useful less as a final paradigm ranking and more as a stress test for few-shot setup, chat templating, and answer-format sensitivity. Table 9 and Figure 4 show that diffusion instruct models are strong in zero-shot mode, while autoregressive instruct models benefit more from few-shot examples and from native chat templating.

Table 7: GSM8K base 0-shot pass@ k , $n = 128$, temperature = 0.7. Values are decimals from the JSON source.

Model	1	2	4	8	16	32	64	128
LLaDA-Base	0.631348	0.750202	0.827433	0.880105	0.916543	0.942333	0.963669	0.984375
Dream-Base	0.723877	0.828404	0.888997	0.923459	0.944172	0.958031	0.968513	0.976562
Qwen-Base	0.709290	0.858601	0.928969	0.963688	0.985947	0.997176	0.999878	1.0
Llama-Base	0.143311	0.258714	0.430480	0.635978	0.817949	0.930355	0.980936	1.0

The zero-shot instruct table is the clearest GSM8K prompt-sensitivity result. LLaDA-Instruct and Dream-Instruct begin at 0.748535 and 0.733887 pass@1, while Qwen-

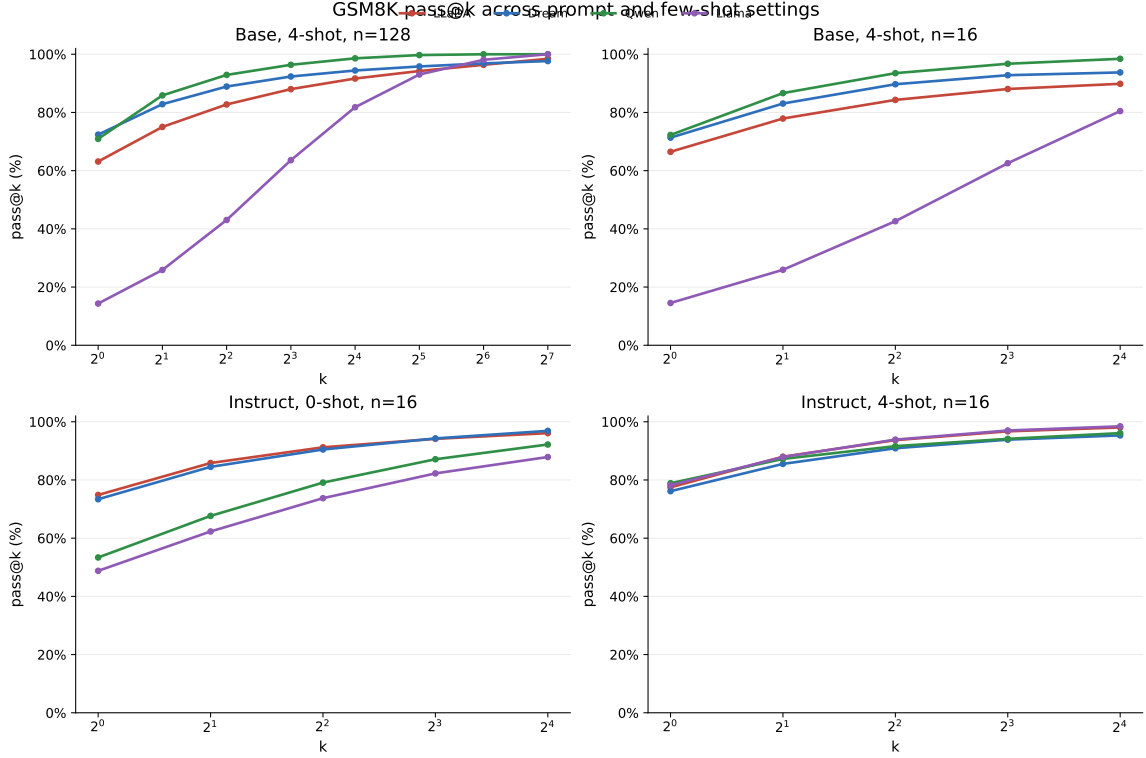


Figure 3: GSM8K pass@k curves across base and instruct settings. The benchmark saturates at high k , but zero-shot instruct behavior differs strongly by paradigm.

Table 8: GSM8K base 4-shot pass@k, $n = 16$, temperature = 0.7.

Model	1	2	4	8	16
LLaDA-Base	0.664795	0.779036	0.843074	0.880577	0.898438
Dream-Base	0.713379	0.830599	0.896920	0.927974	0.937500
Qwen-Base	0.722656	0.866374	0.934980	0.967295	0.984375
Llama-Base	0.145264	0.259310	0.426238	0.625528	0.804688

Table 9: GSM8K instruct pass@k at 0-shot and 4-shot, $n = 16$, temperature = 0.7.

Setting	Model	1	2	4	8	16
0-shot	LLaDA-Instruct	0.748535	0.858268	0.912251	0.941302	0.960938
0-shot	Dream-Instruct	0.733887	0.845085	0.904700	0.942744	0.968750
0-shot	Qwen-Instruct	0.533691	0.676562	0.790973	0.871220	0.921875
0-shot	Llama-Instruct	0.487793	0.623079	0.737427	0.822580	0.878906
4-shot	LLaDA-Instruct	0.774658	0.879883	0.936888	0.967083	0.980469
4-shot	Dream-Instruct	0.761475	0.855241	0.908982	0.938484	0.953125
4-shot	Qwen-Instruct	0.788818	0.872298	0.916204	0.941368	0.960938
4-shot	Llama-Instruct	0.781982	0.878809	0.938661	0.970016	0.984375

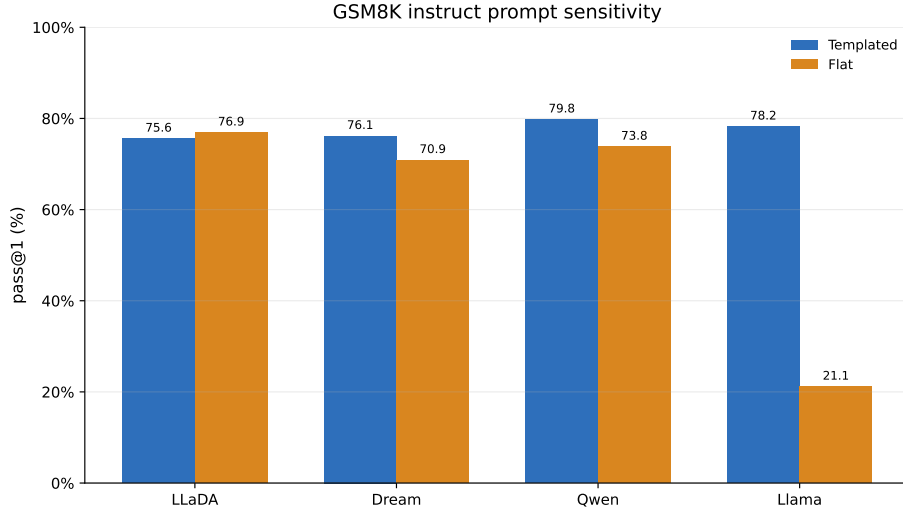


Figure 4: GSM8K instruct prompt-format sensitivity at pass@1, $n = 128$. Llama-Instruct is especially sensitive to flattening, while LLaDA-Instruct slightly improves under the flat format in this run.

Table 10: GSM8K instruct templated versus flat pass@k, $n = 128$, temperature = 0.7.

Model / prompt	1	2	4	8	16	32	64	128
LLaDA-Instruct templated	0.755676	0.852077	0.905127	0.941567	0.967573	0.982231	0.989548	0.992188
LLaDA-Instruct flat	0.769043	0.860548	0.907841	0.936989	0.955312	0.965886	0.974396	0.984375
Dream-Instruct templated	0.761475	0.843946	0.893162	0.926146	0.949600	0.966063	0.974896	0.976562
Dream-Instruct flat	0.708679	0.819758	0.884776	0.922303	0.947388	0.964927	0.977805	0.984375
Qwen-Instruct templated	0.797729	0.875245	0.918452	0.944926	0.965338	0.981428	0.993233	1.0
Qwen-Instruct flat	0.737976	0.828870	0.889439	0.931290	0.959488	0.976366	0.983566	0.984375
Llama-Instruct templated	0.782166	0.882475	0.941822	0.976522	0.993784	0.999108	0.999976	1.0
Llama-Instruct flat	0.211426	0.364816	0.569077	0.776633	0.921081	0.982950	0.998265	1.0

Instruct and Llama-Instruct begin at 0.533691 and 0.487793. With 4-shot prompting, Qwen and Llama rise to 0.788818 and 0.781982 pass@1, essentially closing the gap. Thus GSM8K supports the broader thesis by showing that prompt format and few-shot examples are not minor implementation details.

5.2. RQ1 and RQ2: Countdown-cd4 ranking reversal under pass@k

RQ1 asks whether diffusion models are better in low-sample planning, and RQ2 asks whether larger sampling budgets reverse that answer. Countdown-cd4 provides the clearest evidence that they do: diffusion models lead at pass@1, but autoregressive models overtake decisively by pass@128. Figure 5 and Table 11 are the core empirical answer to the thesis question.

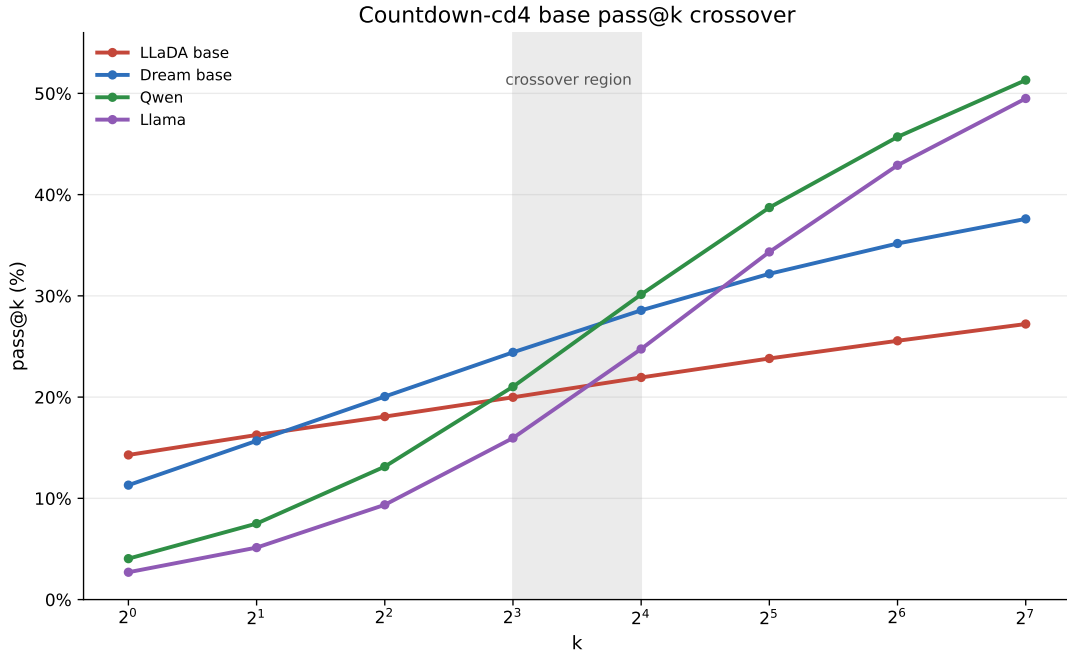


Figure 5: Countdown-cd4 base pass@k crossover. Diffusion models lead at pass@1; Qwen and Llama overtake by large k .

Table 11: Countdown-cd4 base pass@k, $n = 128$, temperature = 0.7.

Model	1	2	4	8	16	32	64	128
LLaDA-Base	0.142854	0.162592	0.180763	0.199806	0.219397	0.238147	0.255617	0.272177
Dream-Base	0.113037	0.156778	0.200547	0.244207	0.285698	0.321771	0.351702	0.376008
Qwen-Base	0.040433	0.075074	0.131354	0.210272	0.301421	0.387202	0.456992	0.513105
Llama-Base	0.027005	0.051350	0.093610	0.159564	0.247519	0.343385	0.428966	0.494960

At pass@1 the ordering is LLaDA-Base > Dream-Base > Qwen-Base > Llama-Base. At pass@128 it becomes Qwen-Base > Llama-Base > Dream-Base > LLaDA-Base. This reversal is not a small fluctuation: Qwen rises from 0.040433 to 0.513105, while

LLaDA rises from 0.142854 to only 0.272177. The result supports the concentration-versus-diversity hypothesis.

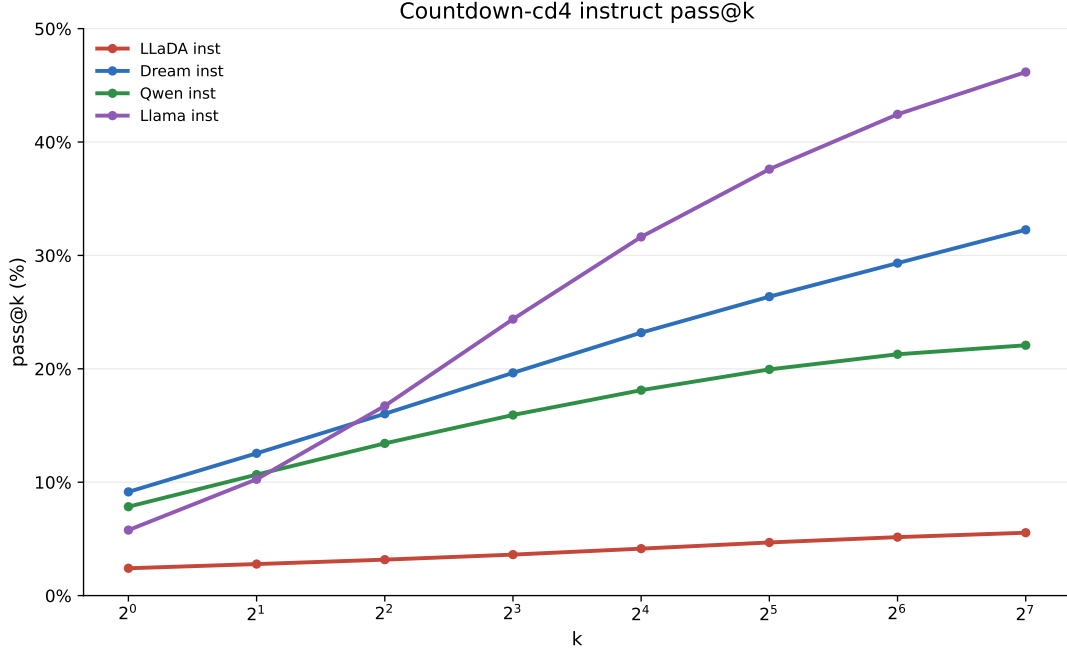


Figure 6: Countdown-cd4 instruct pass@k under templated prompting. Dream-Instruct is the strongest low- k instruct model, while Llama-Instruct becomes strongest at high k .

Table 12: Countdown-cd4 instruct-templated pass@k, $n = 128$, temperature = 0.7.

Model	1	2	4	8	16	32	64	128
LLaDA-Instruct	0.024091	0.027805	0.031700	0.036142	0.041394	0.046858	0.051550	0.055444
Dream-Instruct	0.091482	0.125477	0.160331	0.196433	0.231875	0.263635	0.293203	0.322581
Qwen-Instruct	0.078385	0.106662	0.134234	0.159261	0.181153	0.199472	0.212810	0.220766
Llama-Instruct	0.057751	0.102571	0.167304	0.243821	0.316324	0.376062	0.424427	0.461694

Table 13: Countdown prompt-mode diagnostic from the earlier smaller run.

Model / prompt	pass@1	pass@128
LLaDA-Base	0.028503	0.068359
LLaDA-Instruct templated	0.027954	0.064453
LLaDA-Instruct flat	0.028442	0.050781
Dream-Base	0.019302	0.103516
Dream-Instruct templated	0.012512	0.066406
Dream-Instruct flat	0.018097	0.097656
Qwen-Base	0.008514	0.25
Qwen-Instruct templated	0.007095	0.037109
Qwen-Instruct flat	0.010483	0.119141
Llama-Base	0.028549	0.511719
Llama-Instruct templated	0.062790	0.480469
Llama-Instruct flat	0.049896	0.484375

The instruct setting is more model-dependent than the base setting. Dream-Instruct remains useful at pass@1 and pass@128, Llama-Instruct becomes the strongest high- k

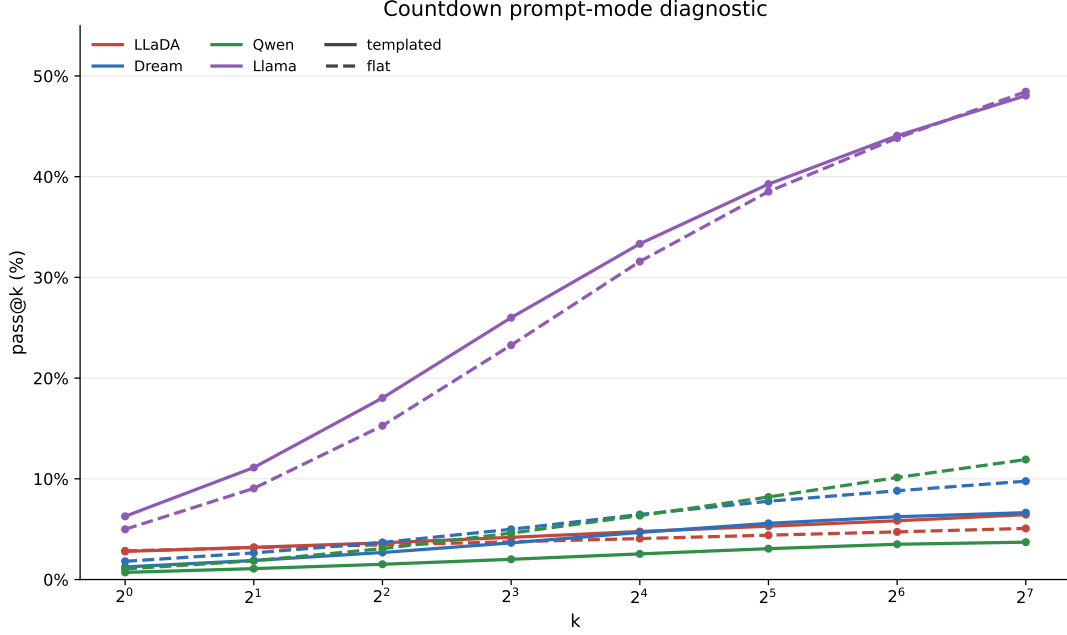


Figure 7: Countdown prompt-mode diagnostic from the earlier smaller run. Flat prompting changes Dream and Qwen substantially and leaves Llama-Instruct strong at high k .

model, and LLaDA-Instruct is weak throughout the curve. This reinforces the methodological point that prompt mode and instruction tuning can reverse conclusions.

5.3. RQ1 and RQ2: Trip Planning as a natural-language planning check

RQ1 and RQ2 should not be answered from arithmetic planning alone. Trip Planning therefore tests whether the Countdown crossover persists under a stricter natural-language parser. The answer is yes in weaker but still meaningful form: Dream remains strongest among the standard diffusion runs at low k , while Llama benefits more from search and the flat-prompt Llama run becomes the strongest high- k curve in the merged comparison of available runs.

Dream-Instruct and Dream-Base are the strongest standard diffusion curves at low k , while Llama-Instruct flat reaches 0.295 at pass@64. This is not identical to Countdown because the flat prompt changes Llama substantially, but the general evaluation lesson is the same: single-sample and many-sample evaluations answer different questions.

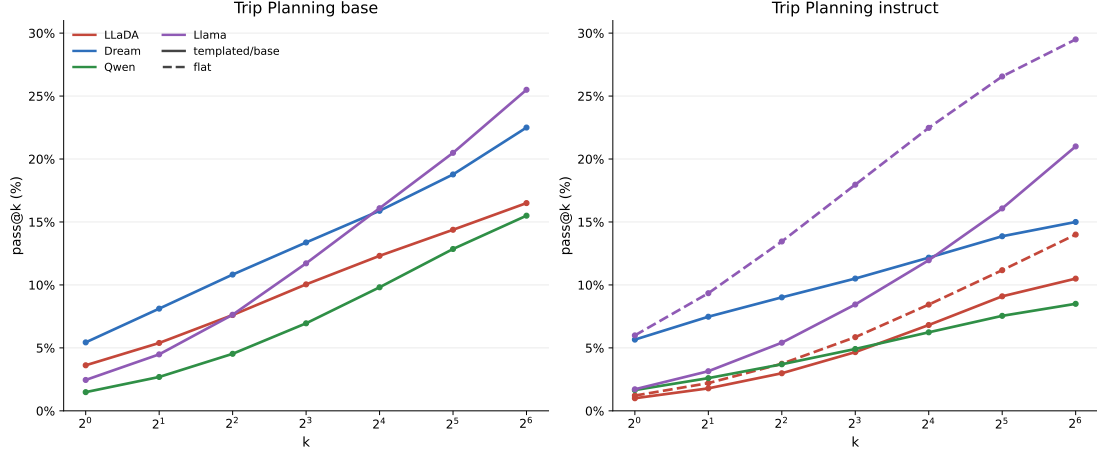


Figure 8: Trip Planning pass@k for all stochastic milestone runs. Dream is strong at low k ; Llama-Instruct flat has the strongest high- k curve.

Table 14: Trip Planning deterministic comparison from accuracy_trip_planning_comparison.json.

Model / variant	Accuracy
Dream-v0-Instruct-7B [bl=256] [dllm]	0.15
Dream-v0-Instruct-7B [bl=32] [dllm]	0.15
Dream-v0-Instruct-7B [bl=256] [fast-dllm]	0.145
Dream-v0-Base-7B [bl=256] [dllm]	0.145
Dream-v0-Base-7B [bl=32] [dllm]	0.145
Dream-v0-Base-7B [bl=32] [fast-dllm]	0.135
Dream-v0-Instruct-7B [flat] [bl=32] [fast-dllm]	0.125
Dream-v0-Instruct-7B [bl=32] [fast-dllm]	0.125
LLaDA-8B-Base [bl=256] [dllm]	0.115
Dream-v0-Instruct-7B [flat] [bl=256] [fast-dllm]	0.11
Dream-v0-Base-7B [bl=256] [fast-dllm]	0.11
Llama-3.1-8B-Instruct [flat] [ar]	0.095
Llama-3.1-8B-Instruct [flat] [vllm]	0.08
LLaDA-8B-Instruct [bl=256] [dllm]	0.08
LLaDA-8B-Base [bl=32] [dllm]	0.075
LLaDA-8B-Base [bl=256] [fast-dllm]	0.06
LLaDA-8B-Base [bl=32] [fast-dllm]	0.05
Llama-3.1-8B [ar]	0.045
Dream-v0-Instruct-7B [flat] [bl=256] [dllm]	0.045
Dream-v0-Instruct-7B [flat] [bl=32] [dllm]	0.045
LLaDA-8B-Instruct [bl=32] [dllm]	0.045
Llama-3.1-8B [vllm]	0.025
LLaDA-8B-Instruct [flat] [bl=32] [dllm]	0.025
Llama-3.1-8B-Instruct [ar]	0.02
Llama-3.1-8B-Instruct [vllm]	0.02
Qwen2.5-7B-Instruct [flat] [ar]	0.02
Qwen2.5-7B-Instruct [flat] [vllm]	0.02
Qwen2.5-7B-Instruct [ar]	0.02
Qwen2.5-7B [ar]	0.02
LLaDA-8B-Instruct [bl=256] [fast-dllm]	0.02
Qwen2.5-7B-Instruct [vllm]	0.015
Qwen2.5-7B [vllm]	0.015
LLaDA-8B-Instruct [flat] [bl=256] [dllm]	0.01
LLaDA-8B-Instruct [flat] [bl=32] [fast-dllm]	0.005
LLaDA-8B-Instruct [flat] [bl=256] [fast-dllm]	0.0
LLaDA-8B-Instruct [bl=32] [fast-dllm]	0.0

Table 15: Trip Planning pass@k, merged from Dream/Qwen, refreshed LLaDA, and Llama milestone sources.

Model	1	2	4	8	16	32	64
Dream-Base	0.054375	0.081171	0.108229	0.133728	0.158874	0.187752	0.225
Dream-Instruct	0.056563	0.074747	0.090129	0.105060	0.121656	0.138662	0.15
LLaDA-Base	0.036172	0.053909	0.076093	0.100449	0.123108	0.143807	0.165
LLaDA-Instruct	0.010000	0.017852	0.029847	0.046623	0.068156	0.090958	0.105
LLaDA-Instruct flat	0.012109	0.022006	0.037392	0.058509	0.084393	0.111698	0.14
Qwen-Base	0.014844	0.026897	0.045297	0.069521	0.098128	0.128546	0.155
Qwen-Instruct	0.016484	0.025977	0.036972	0.049130	0.062370	0.075451	0.085
Llama-Base	0.024531	0.044851	0.076327	0.117150	0.160924	0.204920	0.255
Llama-Instruct	0.017109	0.031468	0.054109	0.084477	0.119603	0.160770	0.21
Llama-Instruct flat	0.059922	0.093423	0.134510	0.179624	0.224752	0.265577	0.295

5.4. RQ1 control: hyperparameter analysis before paradigm comparison

Before the thesis pivoted from tuning to paradigm comparison, the repository ran a GSM8K sweep over generation length, diffusion steps, and block length for LLaDA and Dream, plus generation-length sweeps for Qwen. The sweep confirms that tuning matters, but it does not erase the paradigm-level pass@k question because even well-tuned diffusion runs and autoregressive runs change rank under different sampling budgets.

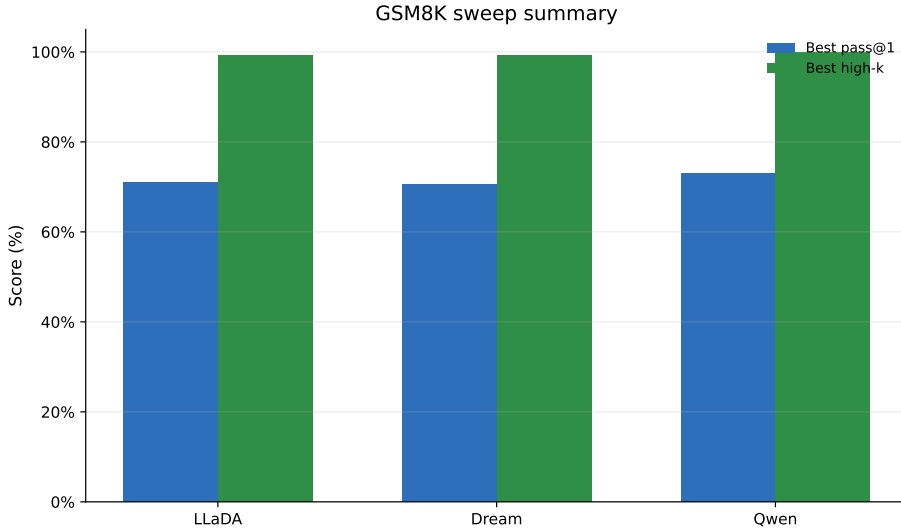


Figure 9: GSM8K sweep summary: best pass@1 and best high-k values. Several diffusion configurations tie at pass@128, so the table reports representative tied configurations.

Generation length is the most important practical parameter in the sweep: 256-token generations dominate the strongest configurations. Diffusion step count has diminishing returns, and block length interacts with both quality and latency. The main

Table 16: Best GSM8K sweep configurations recomputed from raw generation JSONs.

Model	Configuration file	pass@1	pass@128	Wall time (min)
LLaDA best pass@1	GSAI-ML_LLaDA-8B-Base_256_256_128_0.7_4_128_128_generations_fast_d1lm.json	0.710083	0.976562	14.541
LLaDA best pass@128 tie	GSAI-ML_LLaDA-8B-Base_256_128_256_0.7_4_128_128_generations_fast_d1lm.json	0.705322	0.992188	11.670
Dream best pass@1 / pass@128 tie	Dream-org_Dream-v0-Base-7B_256_256_32_0.7_4_128_128_generations_fast_d1lm.json	0.706726	0.992188	12.072
Dream pass@128 tie	Dream-org_Dream-v0-Base-7B_256_64_32_0.7_4_128_128_generations_fast_d1lm.json	0.706604	0.992188	11.388
Qwen best pass@1 / pass@128	Qwen_Qwen2.5-7B_256_0.7_4_128_128_generations_v1lm.json	0.729675	1.000000	5.719

thesis therefore does not attempt to declare a universally best diffusion hyperparameter setting; it asks how the paradigms compare under controlled, documented configurations.

5.5. RQ5: Parser sensitivity and failure rates

RQ5 asks how much of the measured story depends on the parser layer. The high-level answer is benchmark-dependent. On GSM8K and Countdown, outright parser crashes are almost nonexistent; the more important issue is output-format compliance. On Trip Planning, malformed plans can dominate the error profile, especially for base prompts and for Llama-Instruct under the templated prompt. Figure 10 and Table 17 summarize the main rates; the full benchmark-model breakdowns are retained in the appendix CSV exports.

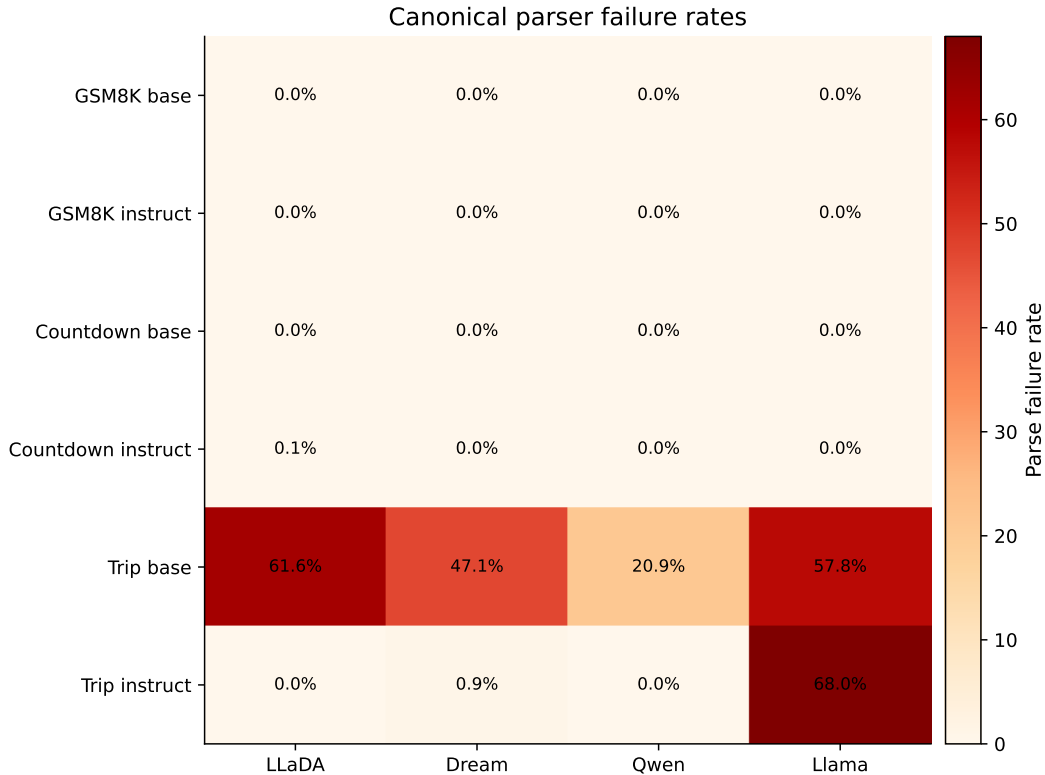


Figure 10: Canonical parser failure rates for the six main comparison conditions. GSM8K and Countdown almost never fail at the parser-entry stage, while Trip Planning can be dominated by malformed outputs.

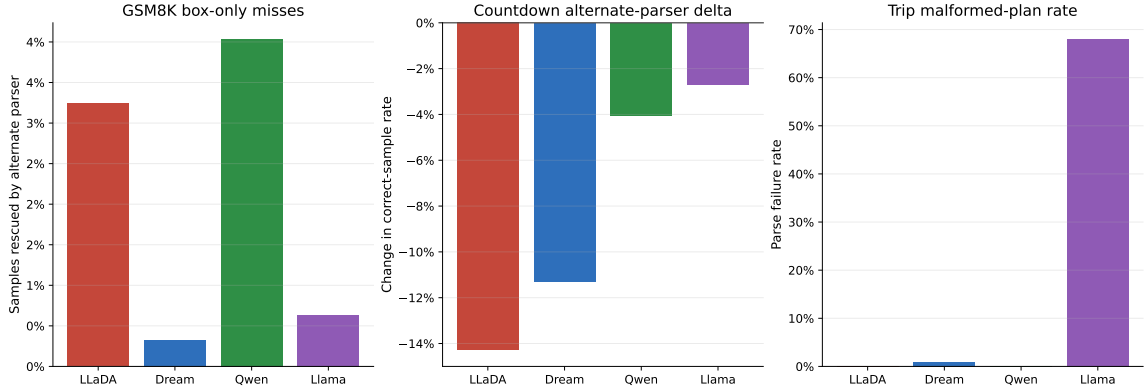


Figure 11: Parser sensitivity diagnostics. Left: GSM8K samples rescued by an alternate final-number fallback. Middle: Countdown alternate expression parser contributes essentially no rescued correct samples. Right: malformed-plan rates remain large for several Trip Planning runs.

Table 17: High-level parser summary. Rates are percentages over all samples in the canonical JSON runs.

Condition	Avg. dLLM parse-fail	Avg. AR parse-fail	Main observation
GSM8K base	0.01	0.02	Parser crashes are rare; the bigger issue is box-format mismatch, especially for Qwen and Llama.
GSM8K instruct	0.01	0.00	Alternate numeric fallback would rescue 3.24 percentage points for LLaDA and 4.03 points for Qwen without changing canonical thesis scores.
Countdown base	0.00	0.00	Parsing is robust; the dominant failure is invalid number reuse rather than syntax.
Countdown instruct	0.04	0.00	Same pattern as base: wrong-number-use dominates, not parser crashes.
Trip Planning base	54.37	39.35	Malformed itineraries are common; missing total-day lines dominate LLaDA, Dream, and Llama base outputs.
Trip Planning instruct	0.47	34.00	Chat formatting almost eliminates Dream/LLaDA parse failures but Llama-Instruct templated still fails to produce parseable plans on 68.0% of samples.

The sensitivity analysis is deliberately kept separate from the canonical thesis numbers. On GSM8K, a more permissive fallback parser recovers correct unboxed answers for all four models, but the effect is largest for the autoregressive checkpoints: Qwen gains 229 additional correct samples in the base 128-sample run and 661 in the instruct-templated run, while LLaDA gains 531 in the instruct-templated run. On Countdown, by contrast, an alternate single-expression parser does not rescue any additional correct samples from the canonical files; the main barrier is semantic invalidity, not chain parsing. On Trip Planning, the dominant parser effect is whether the model emits the required itinerary scaffold at all.

5.6. RQ2: Sampling diversity and pass@k growth

RQ2 can be reframed as a question about search behavior: which models turn additional samples into genuinely different valid candidates? Figure 12 and Table 18 show that the planning benchmarks are at least consistent with the concentration-versus-diversity interpretation. Diffusion models often produce fewer distinct normalized answers and smaller pass@k gains, while the strongest autoregressive high- k models maintain far broader output diversity and convert it into steeper pass@k growth.

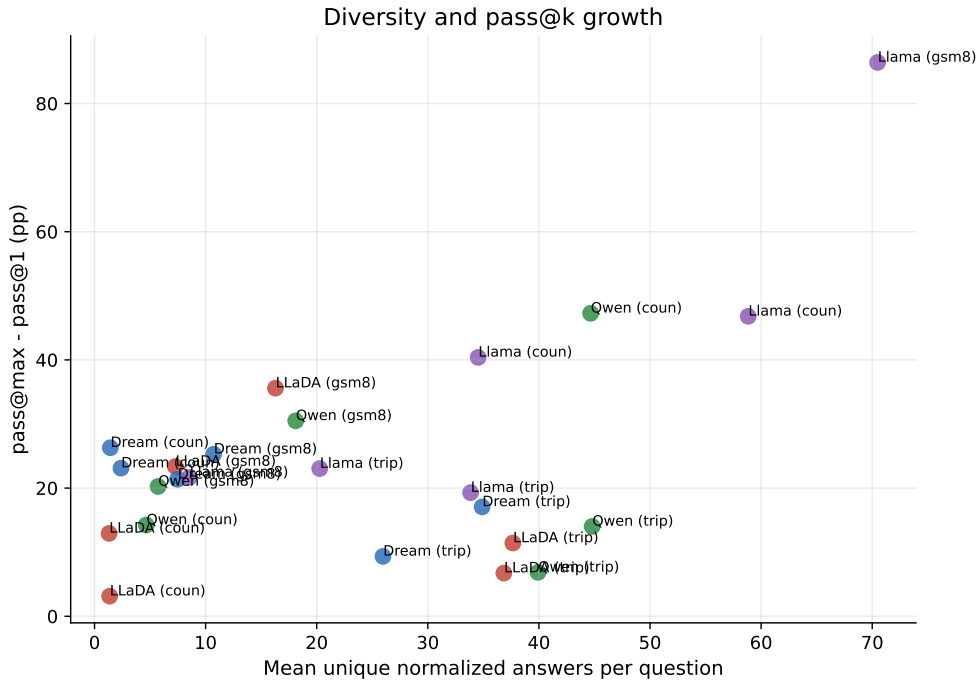


Figure 12: Mean answer diversity versus pass@k gain. The planning-benchmark points are consistent with a concentration-versus-diversity trade-off: broader answer diversity often accompanies larger gains from search.

These metrics do not prove the internal mechanism of diffusion denoising. They do, however, make the thesis interpretation more disciplined. Countdown Base is the

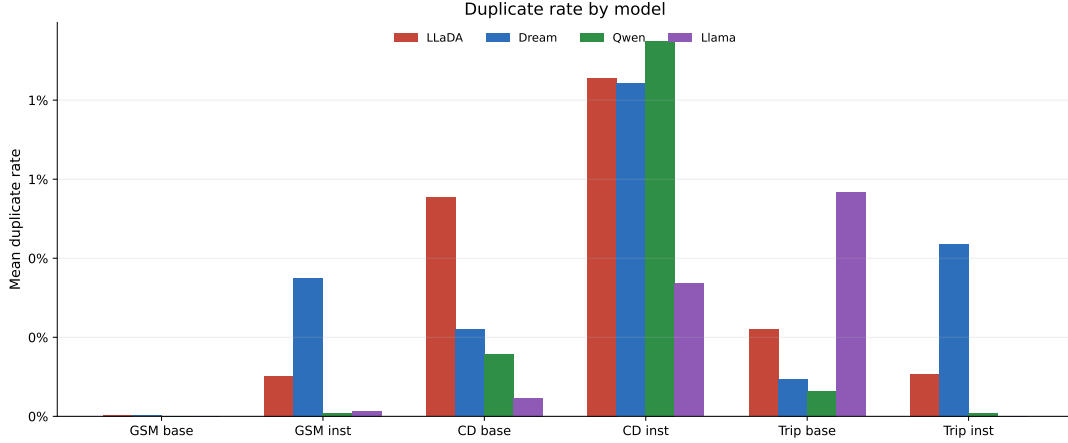


Figure 13: Mean duplicate rate by model and condition. Several diffusion planning runs repeatedly emit near-identical candidates, while the strongest high- k autoregressive runs keep duplicate rates much lower.

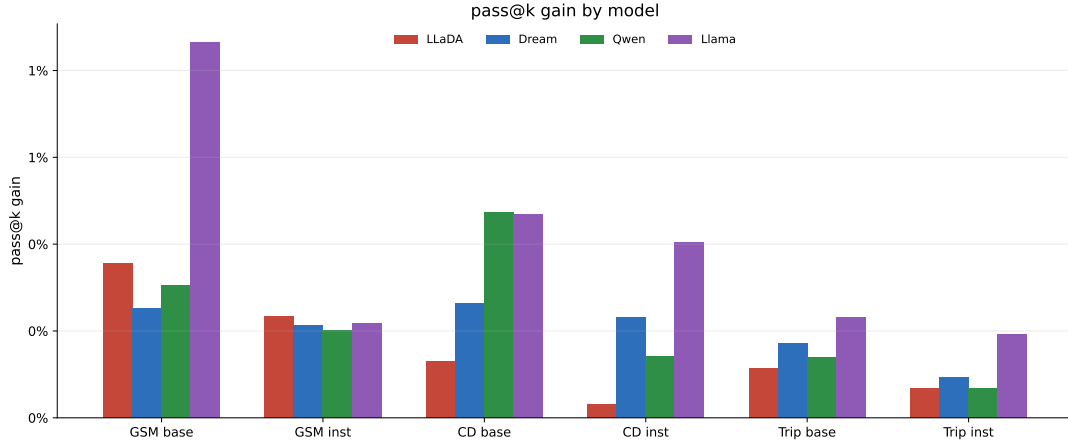


Figure 14: pass@k gain by model and condition. The largest gains appear in the autoregressive planning runs, especially Qwen on Countdown Base and Llama on Trip Planning.

Table 18: Compact diversity summary for the main planning conditions. Unique-answer counts are means per question; gains are pass@max minus pass@1 in percentage points.

Condition	Best dLLM unique answers	Best AR unique answers	Best dLLM gain	Best AR gain
Countdown base	Dream 1.42	Llama 58.84	Dream 26.30	Qwen 47.42
Countdown instruct	Dream 2.39	Llama 34.53	Dream 23.11	Llama 40.14
Trip Planning base	LLaDA 37.66	Qwen 44.79	Dream 17.06	Llama 23.73
Trip Planning instruct	LLaDA 36.84	Qwen 39.92	Dream 9.34	Llama 19.58

clearest case: LLaDA averages only 1.31 unique normalized answers per question and a 12.93-point pass@k gain, while Qwen averages 44.65 unique normalized answers and a 47.27-point gain. Trip Planning is noisier because parser-validity interacts with diversity, but the same broad pattern remains visible.

5.7. RQ4: Per-question correlation and oracle complementarity

RQ4 asks whether the paradigms solve the same questions. The per-question analysis says “partly, but far from completely.” Figure 15 summarizes the within-paradigm and strongest cross-paradigm correlations. Tables 19–24 list the pairwise matrices, and Table 26 adds an oracle-union perspective using the existing samples.

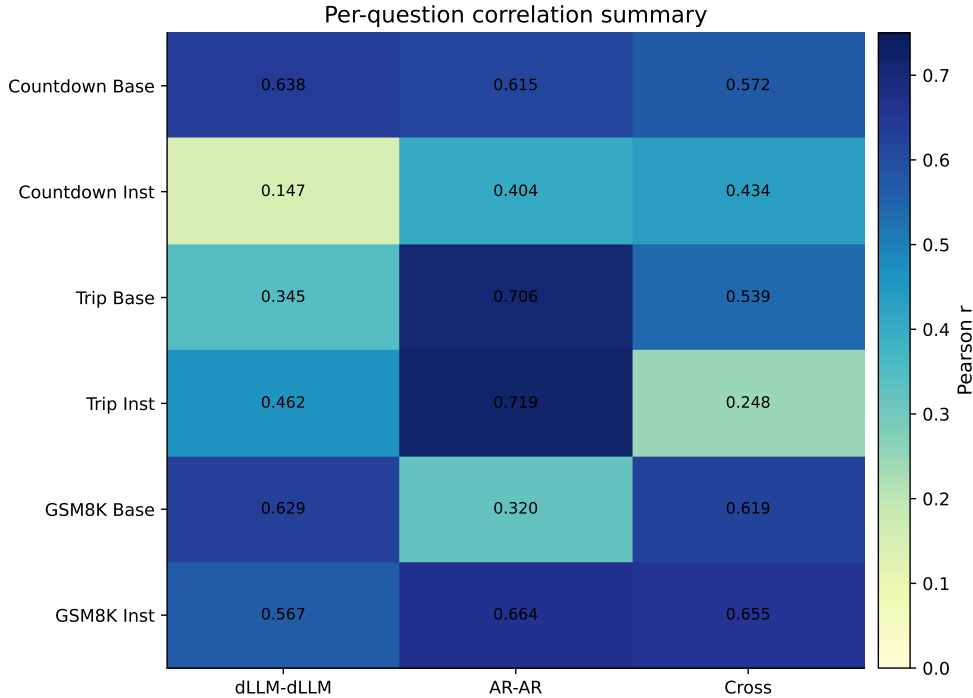


Figure 15: Summary of per-question correlations. Countdown Base has the strongest dLLM–dLLM agreement, while Trip Planning has the strongest AR–AR agreement.

Table 19: Countdown Base per-question correlations.

Model pair	Type	r	p -value
LLaDA-Base vs Dream-Base	dLLM–dLLM	0.6382	1.4×10^{-114}
Qwen-Base vs Llama-Base	AR–AR	0.6149	3.1×10^{-104}
Dream-Base vs Qwen-Base	dLLM–AR, shared backbone lineage	0.5718	3.5×10^{-87}
LLaDA-Base vs Qwen-Base	dLLM–AR	0.5342	2.8×10^{-74}
LLaDA-Base vs Llama-Base	dLLM–AR	0.4580	1.3×10^{-52}
Dream-Base vs Llama-Base	dLLM–AR	0.4432	5.6×10^{-49}

The key result is Countdown Base. LLaDA-Base and Dream-Base correlate at $r = 0.6382$, which is higher than Dream-Base and Qwen-Base at $r = 0.5718$ even

Table 20: Countdown Instruct per-question correlations.

Model pair	Type	r	p -value
Dream-Inst vs Llama-Inst	dLLM-AR	0.4345	6.1×10^{-47}
Dream-Inst vs Qwen-Inst	dLLM-AR	0.4062	1.1×10^{-40}
Qwen-Inst vs Llama-Inst	AR-AR	0.4039	3.3×10^{-40}
LLaDA-Inst vs Dream-Inst	dLLM-dLLM	0.1475	3.1×10^{-6}
LLaDA-Inst vs Llama-Inst	dLLM-AR	0.1464	3.6×10^{-6}
LLaDA-Inst vs Qwen-Inst	dLLM-AR	0.0867	6.3×10^{-3}

Table 21: GSM8K Base per-question correlations.

Model pair	Type	r	p -value
LLaDA-Base vs Dream-Base	dLLM-dLLM	0.6291	1.8×10^{-15}
Dream-Base vs Qwen-Base	dLLM-AR, shared backbone lineage	0.6189	6.9×10^{-15}
LLaDA-Base vs Qwen-Base	dLLM-AR	0.4907	4.1×10^{-9}
LLaDA-Base vs Llama-Base	dLLM-AR	0.4316	3.7×10^{-7}
Dream-Base vs Llama-Base	dLLM-AR	0.3510	4.9×10^{-5}
Qwen-Base vs Llama-Base	AR-AR	0.3204	2.3×10^{-4}

Table 22: GSM8K Instruct per-question correlations.

Model pair	Type	r	p -value
Qwen-Inst vs Llama-Inst	AR-AR	0.6638	1.4×10^{-17}
LLaDA-Inst vs Llama-Inst	dLLM-AR	0.6550	5.0×10^{-17}
Dream-Inst vs Llama-Inst	dLLM-AR	0.6398	4.3×10^{-16}
LLaDA-Inst vs Dream-Inst	dLLM-dLLM	0.5671	3.0×10^{-12}
Dream-Inst vs Qwen-Inst	dLLM-AR, shared backbone lineage	0.5598	6.5×10^{-12}
LLaDA-Inst vs Qwen-Inst	dLLM-AR	0.4847	6.8×10^{-9}

Table 23: Trip Planning Base per-question correlations.

Model pair	Type	r	p -value
Qwen-Base vs Llama-Base	AR-AR	0.7061	1.7×10^{-31}
LLaDA-Base vs Llama-Base	dLLM-AR	0.5394	1.7×10^{-16}
LLaDA-Base vs Qwen-Base	dLLM-AR	0.5158	5.5×10^{-15}
Dream-Base vs Qwen-Base	dLLM-AR, shared backbone lineage	0.4829	4.4×10^{-13}
Dream-Base vs Llama-Base	dLLM-AR	0.3629	1.3×10^{-7}
LLaDA-Base vs Dream-Base	dLLM-dLLM	0.3451	5.6×10^{-7}

Table 24: Trip Planning Instruct per-question correlations.

Model pair	Type	r	p -value
Qwen-Inst vs Llama-Inst	AR-AR	0.7190	4.1×10^{-33}
LLaDA-Inst vs Dream-Inst	dLLM-dLLM	0.4622	5.6×10^{-12}
Dream-Inst vs Llama-Inst	dLLM-AR	0.2484	3.9×10^{-4}
LLaDA-Inst vs Qwen-Inst	dLLM-AR	0.2170	2.0×10^{-3}
LLaDA-Inst vs Llama-Inst	dLLM-AR	0.1647	2.0×10^{-2}
Dream-Inst vs Qwen-Inst	dLLM-AR, shared backbone lineage	0.1572	2.6×10^{-2}

Table 25: Within-paradigm and strongest cross-paradigm correlations.

Condition	dLLM–dLLM	AR–AR	Strongest cross	Interpretation
Countdown Base	0.6382	0.6149	0.5718	Diffusion pair exceeds Dream–Qwen despite Dream’s Qwen lineage.
Countdown Instruct	0.1475	0.4039	0.4345	LLaDA-Instruct is an outlier; Dream aligns more with AR models.
GSM8K Base	0.6291	0.3204	0.6189	Saturated but still shows strong diffusion-pair agreement.
GSM8K Instruct	0.5671	0.6638	0.6550	Instruct prompting makes alignments more mixed.
Trip Planning Base	0.3451	0.7061	0.5394	AR models share the strongest planning-failure pattern.
Trip Planning Instruct	0.4622	0.7190	0.2484	Low cross-paradigm correlations suggest complementarity.

though Dream shares Qwen lineage. This suggests that, for this constrained planning task, generation paradigm is more predictive of failure-mode alignment than shared pretrained architecture. Trip Planning Instruct adds a different insight: cross-paradigm correlations are low, especially Dream-Instruct versus Qwen-Instruct at $r = 0.1572$, which points to complementarity and possible ensemble value.

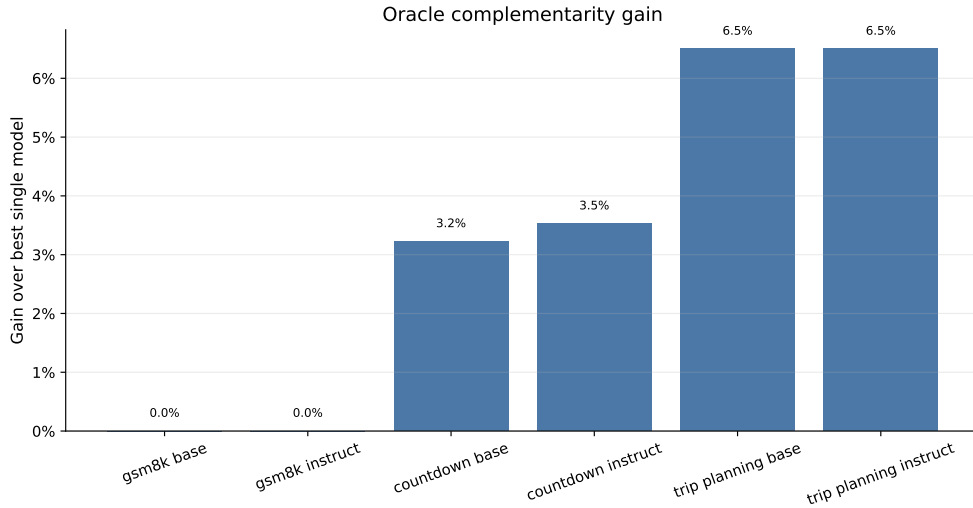


Figure 16: Oracle complementarity gain over the best single model. GSM8K is saturated, but both planning benchmarks show measurable paradigm-level union gains.

The oracle analysis makes the complementarity claim more concrete. On Countdown Base, the best single model solves 509 of 992 questions, but a Dream+Qwen oracle union solves 541 and the full four-model union solves 604. Trip Planning is even more striking relative to its scale: the best single model solves 51 of 200 base questions, while Dream+Llama solves 64 and the full four-model union solves 71. These are only upper bounds because no practical selector is implemented, but they show that the correlation results are not merely statistical decoration.

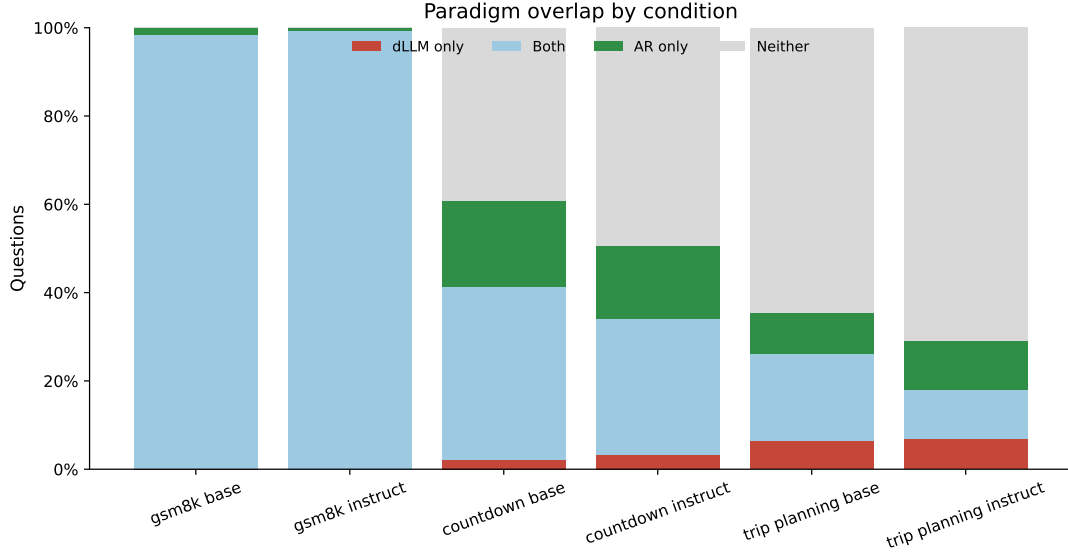


Figure 17: Question overlap between diffusion and autoregressive unions. Trip Planning instruct has especially large “neither” and nontrivial “dLLM only” / “AR only” regions, which is consistent with complementarity rather than mere redundancy.

Table 26: Oracle complementarity from existing samples. Accuracies are percentages of questions solved by at least one sample.

Condition	BestBest dLLM + AR oracle union		All-model oracle union	Gain over best single
	Best	single model		
GSM8K base	Qwen 100.00	LLaDA + Qwen 100.00	100.00	0.00
GSM8K instruct	Qwen 100.00	LLaDA + Qwen 100.00	100.00	0.00
Countdown base	Qwen 51.31	Dream + Qwen 54.54	60.89	3.23
Countdown instruct	Llama 46.17	Dream + Llama 49.70	50.50	3.53
Trip Planning base	Llama 25.50	Dream + Llama 32.00	35.50	6.50
Trip Planning instruct	Llama 21.00	Dream + Llama 27.50	29.00	6.50

6. Discussion

The results support a consistent but carefully limited theory: diffusion LLMs appear to concentrate probability mass through iterative denoising, giving them stronger deterministic and low- k planning behavior, while autoregressive models appear to maintain broader sampling diversity and therefore improve faster under pass@ k . The new diversity metrics make that reading more defensible, especially on Countdown-cd4, where diffusion models emit very few distinct normalized answers per question and AR models emit many more. Even so, the evidence is only *consistent with* this mechanism; it does not directly observe internal probability mass.

The thesis therefore rejects a single global answer to the question “which paradigm is better?” Diffusion models are better when the evaluation permits only one or a few samples, especially on constrained planning. Autoregressive models are better when the evaluation permits many samples and a parser can search over them. The scientific conclusion depends directly on the evaluation protocol.

Prompting is also part of the result rather than a nuisance variable. GSM8K zero-shot instruct favors the diffusion models, but 4-shot prompting lets Qwen and Llama catch up. Countdown and Trip Planning show that flat versus templated prompting can move individual models substantially, sometimes enough to alter qualitative claims. The implication is methodological: prompt mode, chat templating, and few-shot count should be reported alongside model family and backend.

The parser analysis adds a more sobering lesson. GSM8K and Countdown are not dominated by parser crashes, but GSM8K scores still depend on a strict boxed-answer convention and Trip Planning scores can be dominated by malformed plans. Parser dependence is therefore a real threat to validity, not a footnote. The canonical thesis numbers should stay tied to the official evaluation path, but any cross-paradigm claim should be read together with the parser-sensitivity appendix.

The correlation and oracle analyses strengthen the complementarity claim. On Countdown Base, diffusion models fail and succeed together more than Dream and Qwen do, despite the shared Dream/Qwen lineage. On Trip Planning Instruct, cross-paradigm correlations are low and the Dream+Llama oracle union improves over the best single model by 6.5 percentage points. This creates a natural future direction: hybrid evaluation or ensemble methods that exploit the different failure modes rather than forcing a single winner.

7. Limitations

The draft has several limitations. First, parser dependence is unavoidable: GSM8K requires boxed answers in the main evaluator, Countdown requires a chained-equation format, and Trip Planning requires parseable itinerary lines. Different parsers could change absolute scores. Second, GSM8K is partly saturated, so it is more useful for prompt sensitivity than for paradigm ranking. Third, inference systems differ: diffusion pass@k uses fast-dLLM and autoregressive pass@k uses vLLM, so speed comparisons are backend-dependent. Fourth, sample counts differ by benchmark: 128 samples are available for GSM8K and Countdown, while Trip Planning uses 64 samples. Fifth, the comparison is limited to 7B–8B checkpoints and does not test larger model scales. Finally, the ensemble interpretation is supported by correlations but has not yet been validated by an explicit ensemble experiment.

8. Conclusion

This thesis draft answers the motivating question conditionally. Diffusion LLMs do not simply replace autoregressive LLMs for reasoning and planning. They change the evaluation trade-off. In deterministic and low- k settings, LLaDA and Dream often produce stronger planning behavior. Under large- k sampling, Qwen2.5 and Llama 3.1 can overtake because autoregressive sampling explores more diverse candidates. The strongest future direction is therefore not a simple paradigm replacement, but hybrid or ensemble evaluation that uses the complementary strengths of both generation mechanisms.

A. Parser Pseudocode

A.1. GSM8K Answer Extraction

```
gold_answer(text):
    match = regex("#### <number>").search(text)
    return float(remove_commas(match.number))

generated_answer(model_output):
    boxed = last_boxed_or_fboxed_expression(model_output)
    if boxed is None:
        return None
```

```

try:
    return float(boxed)
except ValueError:
    return boxed.strip()

is_correct(pred, gold):
    if pred and gold are numeric:
        return abs(pred - gold) < 1e-4
    return normalized_string(pred) == normalized_string(gold)

```

A.2. Countdown Expression Validation

```

score_countdown(input, prediction):
    numbers, target = parse_input(input)
    available = Counter(numbers)
    equations = parse_leading_equations(first_line(prediction))

    for equation in equations:
        left, right = split_at_equals(equation)
        left = normalize_operators(left)
        right_value = parse_number(right)
        if restricted_eval(left) != right_value:
            return false
        consume_operands_from_counter(left, available)
        available[right_value] += 1

    consume_target(target, available)
    return all_counts_zero(available)
        and final_rhs == target

```

A.3. Trip Planning Plan Validation

```

parse_trip_plan(response):
    total_days = first_line_matching("European cities for N days")
    visit_ranges = scan_lines_for_day_ranges(response)
    flights = scan_lines_for("Day D ... from City to City")
    cities = [first_departure_city] + all_arrival_cities
    flight_days = [1] + flight_days + [total_days]

```

```

    stays = differences_between_adjacent_flight_days_plus_one()
    return zip(cities, stays)

score_trip_plan(gold_cities, gold_durations, response):
    parsed = parse_trip_plan(clean_response(response))
    if parsed is empty:
        return false
    return parsed == ordered gold cities and durations

```

B. Inspected Files and Legacy Parser Notes

The main parser and scoring behavior described in Section 4.4 comes from `inference_and_generation.py`, `dataset/gsm8k.py`, `dataset/countdown.py`, `dataset/trip_planning.py`, `metrics/parsers.py`, `metrics/pass_k.py`, and `evaluate_pass_k.py`. Legacy helper code in `utils.py`, `metrics/parse_and_get_acc.py`, and `metrics/parser_json.py` contains alternative boxed-answer, arbitrary-expression, and JSON-style parsers, but those helpers are not the source of the main Milestone 2 pass@k result JSONs.

C. Prompt Templates and Qualitative Examples

C.1. Prompt-mode summary

GSM8K base_native / instruct_flat:

```

[system prompt with boxed-answer requirement]
Question: <question>
Answer:

```

GSM8K instruct_templated:

```

system: "You are a math expert ... Wrap the final answer in \boxed{}."
user/assistant few-shot turns
user: "Question: <question>\nAnswer:"

```

Countdown base_native / instruct_flat:

```

"Given 5 numbers, use +-* / to operate over the first four numbers
to achieve the fifth number."
base suffix: "Output ONLY comma-separated equations ..."

```

instruct suffix: "No words, no explanation, no retries ..."

Trip Planning base_native / instruct_flat:

TASK: ... <few-shot travel-planning prompt> ...

SOLUTION:

Trip Planning instruct_templated:

single user chat turn containing the same TASK ... SOLUTION: prompt

C.2. Benchmark-specific answer requirements

GSM8K:

final answer expected in \boxed{...}

Countdown-cd4:

comma-separated binary equation chain such as

$a+b=c, c-d=e, e*f=target$

Trip Planning:

total-day statement, day ranges, and "Fly from City to City" lines

D. Qualitative Examples

The following compact examples are extracted from the thesis generation JSON files.

D.1. Countdown-cd4 Base

dLLM-only solved Question. 23,10,3,28,39

LLaDA. $23+10=33, 33/3=11, 11+28=39$ Input: 60

Qwen. $23-10=13, 3*28=84, 13+84=97, 39*2=7$

AR-only solved Question. 97,2,59,5,14

Qwen. $97-59=38, 38/2=19, 19-5=14$ Input: 80,4

LLaDA. $97/2=48, 59/5=34, 48-34=14$ Input: 97,

Both paradigms solved Question. 57,61,22,15,41

LLaDA. $57+21=78, 22+15=37, 78-37=41$ Input: 9

Qwen. $61+22=83, 57-15=42, 83-42=41$ Input: 60

Neither paradigm solved Question. 2,52,20,73,57

LLaDA. $52-2=50, 73-20=53, 50+53=103$ Input: 5,

Qwen. $52-2=50, 73-20=53, 50+53=103$ Input: 79

D.2. Trip Planning Base

dLLM-only solved Question. Hamburg**London**Copenhagen**Tallinn**Vilnius**Zurich**My

LLaDA. **Day 1-5:** Arriving in Hamburg and visit Hamburg for 5 days.

Day 5: Fly from Hamburg to London. **Day 5-6:** Visit London for 2 days. **Day 6:** Fly from London to Copenhagen. **Day 6-8:** Visit Copenhagen for 3 da

Qwen. Here is the trip plan for visiting the 7 European cities for 19 days: **Day 1-5:** Arriving in Hamburg and visit Hamburg for 5 days.

Day 5: Fly from Hamburg to London. **Day 5-6:** Visit London for 2 days. **Day 6:**

AR-only solved Question. Athens**Mykonos**Nice**Edinburgh**Seville||5**2**6**2**2

Qwen. Here is the trip plan for visiting the 5 European cities for 13 days: **Day 1-5:** Arriving in Athens and visit Athens for 5 days.

Day 5: Fly from Athens to Mykonos. **Day 5-6:** Visit Mykonos for 2 days. **Day 6:**

LLaDA. **Day 1-5:** Arriving in Athens and visit Athens for 5 days.

Day 5: Fly from Athens to Mykonos. **Day 5-6:** Visit Mykonos for 2 days. **Day 6:** Fly from Mykonos to Edinburgh. **Day 6-7:** Visit Edinburgh for 2 days

Both paradigms solved Question. Riga**Edinburgh**Seville**Paris**Naples||4**7**4**6**6

LLaDA. **Day 1-4:** Arriving in Riga and visit Riga for 4 days. **Day 4:** Fly from Riga to Edinburgh. **Day 4-9:** Visit Edinburgh for 7 days. **Day 9:** Fly from Edinburgh to Seville. **Day 9-12:** Visit Seville for 4 days. *

Qwen. Here is the trip plan for visiting the 5 European cities for 23 days: **Day 1-4:** Arriving in Edinburgh and visit Edinburgh for 7 days. **Day 4:** Fly from Edinburgh to Riga. **Day 4-7:** Visit Riga for 4 days. **Day 7

Neither paradigm solved Question. Riga**Hamburg**Bucharest**Naples**Nice**Reykjavik**

LLaDA. Here is the trip plan for visiting the 10 European cities for 25 days: **Day 1-5:** Arriving in Hamburg and visit Hamburg for 4 days. **Day 5:** Fly from Hamburg to Madrid. **Day 5-9:** Visit Madrid for 5 days. **Day 9:

Qwen. 1. Arriving in Madrid and visit Madrid for 5 days. 2. Fly from Madrid to Reykjavik. 3. Visit Reykjavik for 3 days. 4. Fly from Reykjavik to Prague. 5. Visit Prague for 2 days. 6. Fly from Prague to Florence. 7. Visit Flo

D.3. GSM8K parser-sensitivity example

Question. Sara wants to buy herself a new jacket and 2 pairs of shoes. The jacket she wants costs \$30 and each pair of shoes cost \$20. Sara babysits the neighbor's kids 4 times, earning \$5 e

Raw sample. <reasoning> First, we need to determine how much money Sara earns from babysitting. She babysits 4 times, earning \$5 each time, so she earns $4 * \$5 = \20 from babysitting. Next, we need to determine how much money Sara already has saved. She had \$10 saved bef

D.4. Countdown parser-failure example

Question. 97,2,59,5,14

Raw sample. $97/2=48, 59/5=34, 48-34=14$ Input: 97,

D.5. Trip Planning parser-failure example

Question. Riga**Hamburg**Bucharest**Naples**Nice**Reykjavik**Prague**Florence**Madri

Raw sample. Here is the trip plan for visiting the 10 European cities for 25 days: **Day 1-5:** Arriving in Hamburg and visit Hamburg for 4 days. **Day 5:** Fly from Hamburg to Madrid. **Day 5-9:** Visit Madrid for 5 days. **Day 9:** Fly from Madrid to Nice. **Day 9-12

References

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [3] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [4] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *International Conference on Learning Representations*, 2023.
- [5] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured Denoising Diffusion Models in Discrete State-Spaces,” *Advances in Neural Information Processing Systems*, vol. 34, 2021.
- [6] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li, “Large Language Diffusion Models,” *arXiv preprint arXiv:2502.09992*, 2025.
- [7] J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong, “Dream 7B: Diffusion Large Language Models,” *arXiv preprint arXiv:2508.15487*, 2025.
- [8] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [9] A. Grattafiori *et al.*, “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [10] K. Cobbe *et al.*, “Training Verifiers to Solve Math Word Problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [11] J. Xie, K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, Y. Xiao, and Y. Su, “TravelPlanner: A Benchmark for Real-World Planning with Language Agents,” *arXiv preprint arXiv:2402.01622*, 2024.
- [12] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *arXiv preprint arXiv:2309.06180*, 2023.

E. AI Usage Card

This appendix is included because the provided GIPPLab sample thesis contains an AI usage card. The final declaration should be adjusted to the applicable University of Göttingen and study-program rules before submission.

Table 27: Draft AI usage card for the thesis project.

Category	Draft entry
Tools used	ChatGPT/Codex was used to audit the local evaluation repository, inspect parser and scoring code, generate thesis figures from JSON artifacts, and revise the LaTeX draft prose.
Human responsibility	The author remains responsible for checking every claim, table value, citation, and interpretation before submission.
Data and results	No new experimental numbers were invented. The tables and figures are based on local JSON artifacts, Milestone 2 documents, and inspected repository code.
Writing status	This remains a thesis draft. It now has real figures, parser documentation, and strengthened prose, but it still needs supervisor review and final polishing before submission.